

Évolution de l'équation de Gross-Pitaevskii pour la modélisation de vortex dans un condensat de Bose-Einstein

Moro Valentin

16 mai 2026

Résumé

Ce projet est dédié à la modélisation numérique de la dynamique d'un condensat de Bose-Einstein par l'intermédiaire de l'équation de Gross-Pitaevskii. Ce travail de modélisation numérique s'inscrit dans le cadre de la validation du Projet Python annuel requis en deuxième année de CMI technologies quantiques.

D'autre part, ce travail est essentiel à l'acquisition de compétences nécessaires à mener à bien mon stage d'été, décroché dans l'université de Purdue, West-Lafayette, Indiana dont le sujet sera également la modélisation de condensat de Bose-Einstein. Ainsi, ce projet constitue le socle de base pour pouvoir amorcer ce stage dans les meilleures conditions.

github associé au projet :
https://github.com/Ribzman/Projet_Crank_Nicolson.git

Table des matières

1	Introduction	3
2	Cas d'un condensat Unidimensionnel	3
2.1	Simulations	3
2.1.1	Algorithme de Crank-Nicolson Unidimensionnel	4
2.1.2	Paquet d'onde initial	6
2.2	Traduction en Python	6
2.3	Résultats d'un système Unidimensionnel	8
3	Passage à un système Bidimensionnel	11
3.1	Simulations	11
3.1.1	Algorithme de Crank-Nicolson Bidimensionnel	12
3.1.2	Paquet d'onde initial	13
3.2	Traduction en python	14
3.3	Résultats d'un système Bidimensionnel	16
3.3.1	Vortex solitaire centré en $(0,0)$	16
3.3.2	Deux vortex centrés en $(-2,2)$ et $(2,-2)$	18
4	Conclusion	20
5	Annexes	22
5.1	Annexe 1 : Etablissement de l'algorithme de Crank-Nicolson 1D .	22
5.2	Annexe 2 : Etablissement de l'algorithme de Crank-Nicolson 2D .	25
5.3	Annexe 3 : Simulation de l'équation de Gross-Pitaevskii 1D avec un soliton lumineux	29
5.4	Annexe 4 : Simulation de la l'équation de Gross-Pitaevskii 2D avec le profil de densité de Thomas-Fermi	30

1 Introduction

Prédit en 1925 par le Physicien Albert Einstein et basé sur les travaux du Physicien Satyendranath Bose, le condensat de Bose-Einstein(BEC) est un état de la matière où un groupe de particules (ou certains atomes) occupe, à des températures suffisamment basses, le même état quantique de plus basse énergie.

Les bosons peuplant tous uniquement cet état quantique, le BEC gagne les propriétés d'un superfluide soit celles d'un fluide parfait et incompressible. Dans ce cadre il est possible d'étudier numériquement, et au moyen de l'équation de l'évolution temporelle de la fonction d'onde : l'équation de Gross-Pitaevskii, la variation de la fonction d'onde au cours du temps.

Nous nous pencherons donc dans un premier temps sur une étude de l'équation de Gross-Pitaevskii en une seule dimension et offrirons une première résolution numérique. Puis, nous verrons ce système évoluer en deux dimensions pour enfin observer des Vortex dans ce condensat bidimensionnel. L'étude numérique sera assurée par l'intermédiaire du langage de programmation Python.

2 Cas d'un condensat Unidimensionnel

2.1 Simulations

Nous considérons des atomes froids piégés dans un puits de potentiel harmonique :

$$V(x) = \frac{x^2}{2}$$

L'équation de Gross-Pitaevskii dans le cas unidimensionnel est donnée :

$$i\hbar \frac{\partial \psi(x,t)}{\partial t} = \left[-\frac{\hbar^2}{2m} \frac{\partial^2}{\partial x^2} + V(x) + g_{1D} |\psi(x,t)|^2 \right] \psi(x,t) \quad (1)$$

Pour des raisons de simplicité nous nous placerons dans le cas où $m = \hbar = 1$ l'équation (1) simplifiée donne donc :

$$i \frac{\partial \psi(x,t)}{\partial t} = \left[-\frac{1}{2} \frac{\partial^2}{\partial x^2} + V(x) + g |\psi(x,t)|^2 \right] \psi(x,t) \quad (2)$$

Nous avons, pour la fonction d'onde ψ , le calcul de la norme qui peut s'interpréter comme le nombre de particules :

$$N = \int |\psi|^2 dx$$

Ainsi que l'énergie associée en une dimension :

$$E = \int \left(\frac{1}{2} \left| \frac{\partial \psi}{\partial x} \right|^2 + V(x) |\psi|^2 + \frac{g}{2} |\psi|^4 \right) dx$$

2.1.1 Algorithme de Crank-Nicolson Unidimensionnel

(Voir l'Annexe 1 en 5.1 page 22 pour le détail complet du calcul)

Notre problème unidimensionnel est de la forme :

$$\frac{\partial \psi}{\partial t} = F\left(\psi, x, t, \frac{\partial \psi}{\partial x}, \frac{\partial^2 \psi}{\partial x^2}\right) \quad (3)$$

Nous pouvons donc écrire la discrétisation temporelle et spatiale d'après l'algorithme de Crank-Nicolson :

$$\begin{cases} \frac{\partial \psi}{\partial t} = \frac{\psi_x^{t+\Delta t} - \psi_x^t}{\Delta t} \\ F_x^{t+\Delta t}(\psi, x, t, \frac{\partial \psi}{\partial x}, \frac{\partial^2 \psi}{\partial x^2}) = -\frac{1}{2} \frac{\psi_x^{t+\Delta t} - 2\psi_x^t + \psi_x^{t-\Delta t}}{\Delta x^2} + \left[V(x) + g|\psi|^2 \right] \psi_x^{t+\Delta t} \\ F_x^t(\psi, x, t, \frac{\partial \psi}{\partial x}, \frac{\partial^2 \psi}{\partial x^2}) = -\frac{1}{2} \frac{\psi_x^t - 2\psi_x^t + \psi_x^{t-\Delta t}}{\Delta x^2} + \left[V(x) + g|\psi|^2 \right] \psi_x^t \end{cases}$$

En prenant la moyenne, en posant $r = \frac{\Delta t}{2\Delta x^2}$ et en posant $\tilde{V} = V(x) + g|\psi|^2$ nous avons :

$$\begin{aligned} & -\frac{ir}{2} \psi_{x+\Delta x}^{t+\Delta t} + (1 + ir + \frac{i\Delta t}{2} \tilde{V}) \psi_x^{t+\Delta t} - \frac{ir}{2} \psi_{x-\Delta x}^{t+\Delta t} \\ & = \frac{ir}{2} \psi_{x+\Delta x}^t + (1 - ir - \frac{i\Delta t}{2} \tilde{V}) \psi_x^t + \frac{ir}{2} \psi_{x-\Delta x}^t \end{aligned}$$

Cette expression peut s'écrire sous la forme matricielle telle que :

$$A' \psi_x^{t+\Delta t} = B' \psi_x^t \quad (4)$$

Où A' et B' sont des matrices tridiagonales.

En appliquant l'approximation $|\psi|^2 = \left| \frac{3\psi_x^t - \psi_x^{t-\Delta t}}{2} \right|^2$ sur la densité dans le terme non linéaire nous pouvons scinder les matrices A' et B' telles que :

$$\begin{cases} A' = A + O \\ B' = B - O \end{cases}$$

Ainsi l'équation matricielle (4) s'écrit :

$$(A + O)\psi_x^{t+\Delta t} = (B - O)\psi_x^t$$

Il faudra donc pour chaque itération, uniquement calculer la matrice O et les matrices A et B seront fixes. Il suffira donc d'inverser cette matrice pour trouver l'itération $t + \Delta t$ de ψ . Nous gagnons donc en rapidité de calcul. Cependant ce choix d'approximation n'est pas le plus optimal quand les matrices deviennent très grandes. Nous perdons en vitesse de calcul et de simulation au profit d'une meilleure approximation physique.

Pour vérifier du bon fonctionnement de l'algorithme nous veillerons aussi à ce que la norme de la fonction d'onde ψ et l'énergie soit des quantités conservées.

L'autre travail de développement de cet algorithme va être de définir les pas d'espace et de temps.

1. Nous définissons une longueur d'espace de 40 et on choisit de prendre 500 valeurs de x dans cet intervalle nous aurons donc un pas d'espace initial de $\Delta t = \frac{L}{N_x} = \frac{40}{500}$ nous définissons pour le pas de temps une quantité appelé "healing lenght", sans dimension nous avons :

$$\begin{aligned} \frac{1}{2\xi^2} &= gn \\ \Rightarrow \xi &= \frac{1}{\sqrt{2gn}} = \frac{cte}{\sqrt{g}} \end{aligned}$$

Nous pouvons maintenant essayer de trouver une valeur de g pour laquelle le programme est fonctionnel au pas d'espace défini initialement et nous trouvons que $g = 1$ fonctionne. En changeant maintenant le facteur g pour lui mettre une valeur quelconque, nous devons avoir en tout temps :

$$\begin{cases} \xi_1 = \Delta x_1 \\ \xi_2 = \Delta x_2 \end{cases}$$

Nous égalisons :

$$\begin{aligned} \Rightarrow \Delta x_2 &= \frac{\xi_2}{\xi_1} \Delta x_1 \\ \Rightarrow \Delta x_2 &= \sqrt{\frac{1}{g}} \frac{L}{N_x} \end{aligned}$$

Nous avons donc le pas pour un instant quelconque : $\Delta x = \sqrt{\frac{1}{g}} \frac{L}{N_x}$

2. Pour le pas de temps nous nous fixons sur les conditions de Courant-Friedrichs-Lewy(CFL) qui nous renseignent que :

$$\Delta t \leq \frac{\Delta x^2}{2}$$

$$\text{soit } \Delta t \leq \frac{\frac{1}{g} \frac{L^2}{N_x^2}}{2}$$

$$\implies \Delta t \leq \frac{1}{g} \frac{L^2}{2N_x^2}$$

Nous avons donc : $\Delta t \leq \frac{1}{g} \frac{L^2}{2N_x^2}$ et $\Delta x = \sqrt{\frac{1}{g} \frac{L}{N_x}}$

2.1.2 Paquet d'onde initial

Comme profil d'onde initial nous allons nous fixer sur une gaussienne de la forme :

$$\psi(x_0, t_0) = \psi_0 e^{-\frac{x^2}{2\sigma^2}} \quad (5)$$

Où σ est l'incertitude sur la position ou l'étalement du condensat.

2.2 Traduction en Python

Nous définissons les matrices A et B

```

1  """On definit les diagonales de A (Inferieure, Principale et Superieure)
2  puis on construit la matrice tridiagonale au format Compressed Sparse Column
3  pour plus de rapidité de calcul"""
4  Principale_DiagA = (1 + 1j*r + 0.5j*dt*V) * np.ones(Nx)
5  Diags_Inf_SuppA = -0.5j*r * np.ones(Nx-1)
6  A = diags([Diags_Inf_SuppA, Principale_DiagA, Diags_Inf_SuppA], [1,0,-1]).tocsc()
7
8  """On definit les diagonales de B (Inferieure, Principale et Superieure)
9  puis on construit la matrice tridiagonale au format Compressed Sparse Column
10 pour plus de rapidité de calcul"""
11 Principale_DiagB = (1 - 1j*r - 0.5j*dt*V) * np.ones(Nx)
12 Diags_Inf_SuppB = 0.5j*r * np.ones(Nx-1)
13 B = diags([Diags_Inf_SuppB, Principale_DiagB, Diags_Inf_SuppB], [1,0,-1]).tocsc()

```

Nous définissons les fonctions de calcul de norme, d'énergies et de la construction de la matrice O :

```

1  """Fonction calculant la norme de Psi"""
2  def calculate_norm(psi):

```

```

3     return np.sum(np.abs(psi)**2 * dx) #Calcul de la norme
4
5     """Fonction calculant l'energie du condensat"""
6     def calculate_energy(psi, psi_prev):
7         psi_extrap = 1.5 * psi - 0.5 * psi_prev #Calcul de Psi extrapolée
8         extrap_density = np.abs(1.5 * psi - 0.5 * psi_prev)**2 #Calcul de la densité extrapolée
9
10        grad2_psi = np.gradient(np.gradient(psi_extrap, dx), dx) #Calcul du gradient 2 ème de psi
11        grad_density = -np.real(np.conj(psi_extrap) * grad2_psi) #Calcul du terme en densité gradient
12
13        current_energy = np.sum((1/2 * grad_density + V * extrap_density
14                                + g/2 * extrap_density**2)* dx)
15        #Calcul de l'énergie
16        return current_energy
17
18    """Fonction qui construit la Matrice O"""
19    def Construct_O(psi, psi_prev):
20        extrap_density = np.abs((1.5 * psi - 0.5 * psi_prev))**2
21        #Calcule la densité a partir de l'extrapolation.
22
23        """Calcule la matrice diagonale et la convertit
24        en Compressed Sparse Column pour plus de rapidité de calcul"""
25        Principale_Diag0 = 0.5j*dt*g*extrap_density * np.ones(Nx)
26
27        return diags([Principale_Diag0],[0]).tocsc()

```

Maintenant nous pouvons traduire le corps de l'algorithme

```

1     """Definit la fonction d'animation et le corps d'exécution de l'algorithme"""
2     def animate(i):
3
4         global psi, psi_prev, phys_time
5         """On definit le corps de l'algorithme
6         en construisant la matrice O et en inversant
7         la Matrice A+O construite"""
8         for _ in range(0, steps_per_frame):
9             O = Construct_O(psi, psi_prev)
10
11             Aprime = A + O #Calcule la matrice A prime
12             Bprime = B - O #Calcule la matrice B prime
13             res = Bprime.dot(psi) #On multiplie par la fonction psi
14             psi_next = spsolve(Aprime, res) #On resoud pour l'itération suivante de psi
15
16             """On met a jour psi_prev et psi"""
17             psi_prev = psi.copy()
18             psi = psi_next.copy()
19

```

```

20         phys_time += dt
21
22         """On ajoute à chaque iteration la norme,
23         l'energie et l'instant dans la liste correspondante."""
24         norms.append(calculate_norm(psi))
25         energies.append(calculate_energy(psi, psi_prev))
26         times.append(phys_time)
27
28         """On redimensionne les axes si besoin."""
29         if len(times) > 1:
30             ax2.set_xlim(0, times[-1])
31             ax3.set_xlim(0, times[-1])
32
33         if len(norms) > 1:
34             ax2.set_ylim(min(norms) * 0.9, max(norms) * 1.1)
35
36         if len(energies) > 1:
37             ax3.set_ylim(min(energies) * 0.9, max(energies) * 1.1)
38
39         line.set_ydata(np.abs(psi)**2) #On definit la donnée à utiliser
40         line2.set_data(times[-tmax:], norms[-tmax:])
41         line3.set_data(times[-tmax:], energies[-tmax:])
42         #On definit la donnée à utiliser (tmax premieres normes, énergies et instants)
43         return line, line2, line3 #on retourne les deux courbes.

```

2.3 Résultats d'un système Unidimensionnel

Les animations sont disponibles sur le github associé au projet. Nous lançons le programme pour notre paquet d'onde initial avec une interaction entre les bosons, g à 1.

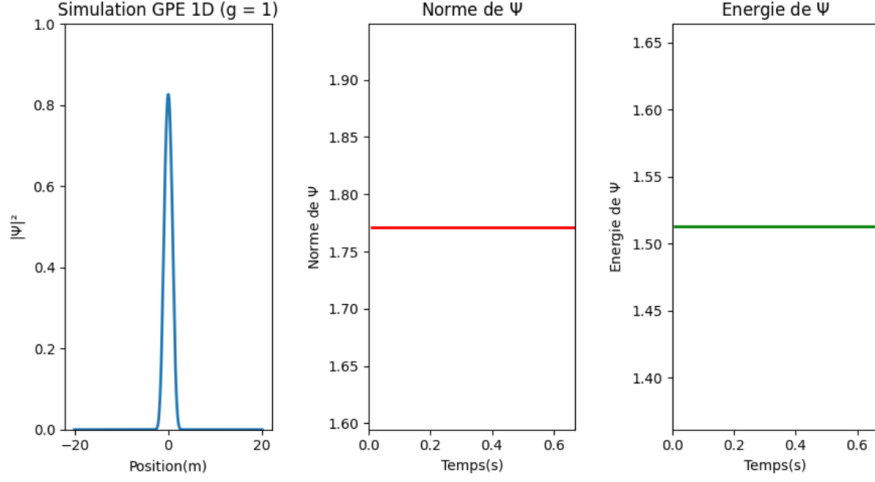


FIGURE 1 – $g = 1$

La norme et l'énergie sont conservées au-cours de la simulation. Nous pouvons maintenant recommencer la simulation avec une interaction entre les "bosons" plus forte et vérifier la conservation de la norme et de l'énergie. Nous prendrons pour cela successivement $g = 10$, $g = 100$ et $g = 400$:

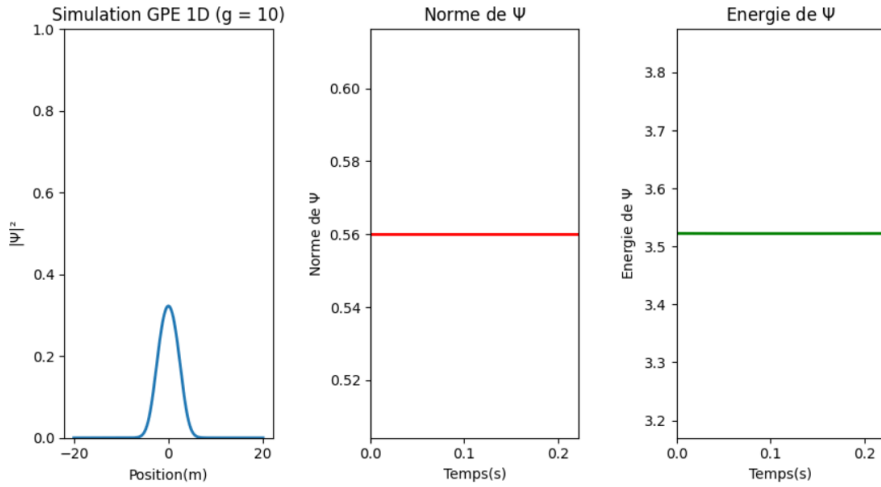


FIGURE 2 – $g = 10$

Le programme reste stable pour une interaction entre "bosons" de 10. L'énergie et la norme restent conservées. La description physique reste valide.

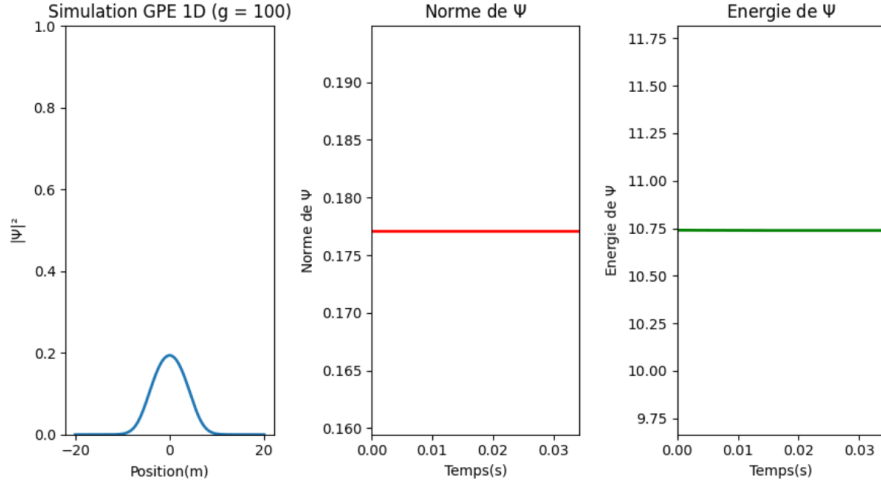


FIGURE 3 – $g = 100$

L'interaction entre "bosons" devient très forte et le condensat subit un étalement important. La norme et l'énergie sont une nouvelle fois conservées.

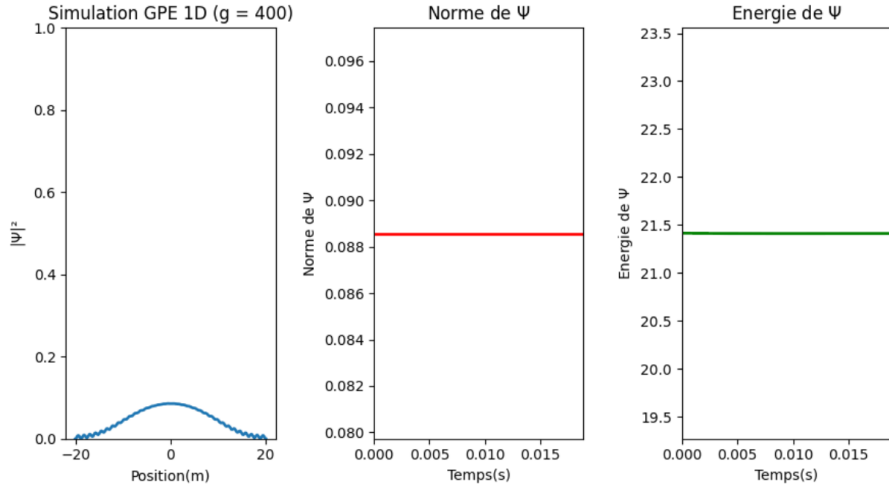


FIGURE 4 – $g = 400$

Ici nous remarquons que $g|\psi|^2\psi \gg -\frac{1}{2}\frac{\partial^2\psi}{\partial x^2} + V(x)\psi$

Ainsi l'interaction entre "bosons" augmente et l'étalement est important. Nous quittons donc progressivement le régime de la gaussienne initiale pour passer à

une forme de parabole inversée.

De plus nous voyons apparaître des oscillations importantes sur les bords du condensat. Ces oscillations peuvent être assimilées au phénomène de Gibbs lié à des effets de bords du système considéré.

3 Passage à un système Bidimensionnel

3.1 Simulations

Dans un potentiel de piégeage bidimensionnel harmonique,

$$V(x, y) = \frac{(x^2 + y^2)}{2}$$

L'équation de Gross-Pitaevskii peut se généraliser pour un système bidimensionnel comme :

$$i \frac{\partial \psi(x, y, t)}{\partial t} = \left[-\frac{1}{2} \nabla^2 + V(x, y) + g |\psi(x, y, t)|^2 \right] \psi(x, y, t) \quad (6)$$

Où ∇^2 est le Laplacien en deux dimensions.

Soit :

$$i \frac{\partial \psi(x, y, t)}{\partial t} = \left[-\frac{1}{2} \left(\frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} \right) + V(x, y) + g |\psi(x, y, t)|^2 \right] \psi(x, y, t) \quad (7)$$

Nous retrouvons aussi le calcul de la norme :

$$N = \iint |\psi(x, y, t)|^2 dx dy$$

Ainsi que l'énergie :

$$E = \iint \left(\frac{1}{2} |\nabla \psi(x, y, t)|^2 + V(x, y) |\psi(x, y, t)|^2 + \frac{g}{2} |\psi(x, y, t)|^4 \right) dx dy$$

Par analogie avec la version unidimensionnelle, l'énergie et la norme sont toujours des quantités conservées. Cependant le moment angulaire est aussi une quantité conservée et s'exprime :

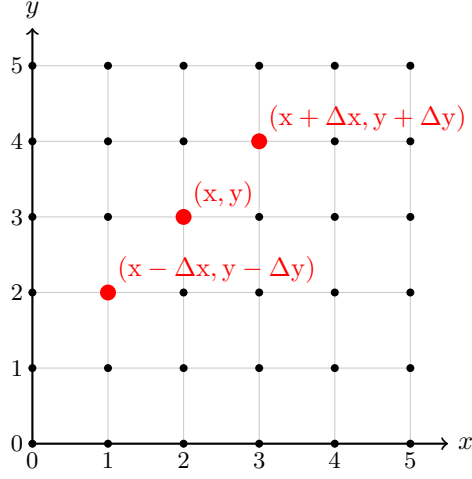
$$L_z = \iint \psi^* [\mathbf{e}_z \cdot (-i \mathbf{x} \times \nabla)] \psi dx dy$$

Pour effectuer cette simulation en deux dimensions on utilise une fois de plus l'algorithme de Crank-Nicolson.

3.1.1 Algorithme de Crank-Nicolson Bidimensionnel

(Voir l'Annexe 2 en 5.2 page 25 pour le détail complet du calcul)

Nous découpons l'espace sous forme d'une grille carrée



Nous pouvons écrire les discrétisations spatiales et temporelles

$$\begin{cases} i \frac{\partial \psi(x, y, t)}{\partial t} = i \frac{\psi_{x, y}^{t+\Delta t} - \psi_{x, y}^t}{\Delta t} \\ \frac{\partial^2 \psi(x, y, t)}{\partial x^2} = \frac{1}{2(\Delta x)^2} \left((\psi_{x+\Delta x, y}^{t+\Delta t} - 2\psi_{x, y}^{t+\Delta t} + \psi_{x-\Delta x, y}^{t+\Delta t}) + (\psi_{x+\Delta x, y}^t - 2\psi_{x, y}^t + \psi_{x-\Delta x, y}^t) \right) \\ \frac{\partial^2 \psi(x, y, t)}{\partial y^2} = \frac{1}{2(\Delta y)^2} \left((\psi_{x, y+\Delta y}^{t+\Delta t} - 2\psi_{x, y}^{t+\Delta t} + \psi_{x, y-\Delta y}^{t+\Delta t}) + (\psi_{x, y+\Delta y}^t - 2\psi_{x, y}^t + \psi_{x, y-\Delta y}^t) \right) \end{cases}$$

En posant

$$\begin{cases} r_x = -\frac{\Delta t}{4i(\Delta x)^2} \\ r_y = -\frac{\Delta t}{4i(\Delta y)^2} \\ \tilde{V} = V(x, y) + g|\psi|^2 \end{cases}$$

Nous trouvons :

$$\begin{aligned} & -r_y \psi_{x, y+\Delta y}^{t+\Delta t} - r_y \psi_{x, y-\Delta y}^{t+\Delta t} + (1 + 2r_y + 2r_x + \frac{\tilde{V}i\Delta t}{2}) \psi_{x, y}^{t+\Delta t} - r_x \psi_{x+\Delta x, y}^{t+\Delta t} - r_x \psi_{x-\Delta x, y}^{t+\Delta t} \\ & = r_y \psi_{x, y+\Delta y}^t + r_y \psi_{x, y-\Delta y}^t + (1 - 2r_y - 2r_x - \frac{\tilde{V}i\Delta t}{2}) \psi_{x, y}^t + r_x \psi_{x+\Delta x, y}^t + r_x \psi_{x-\Delta x, y}^t \end{aligned}$$

Que nous pouvons écrire sous forme matricielle :

$$A' \psi_{x, y}^{t+\Delta t} = B' \psi_{x, y}^t$$

Où A' et B' sont des matrices tridiagonales.

De la même manière que pour le condensat unidimensionnel nous pouvons scinder les matrices A' et B' en utilisant une approximation du terme de la densité dans la partie non linéaire. Nous retrouvons :

$$(A + O)\psi_x^{t+\Delta t} = (B - O)\psi_x^t$$

Matrice qu'il ne reste plus qu'à inverser pour trouver l'itération $t + \Delta t$ de ψ .

Pour le calcul des pas d'espace de notre programme nous reprenons dans les deux directions :

$$\Delta x = \sqrt{\frac{1}{g} \frac{L}{N_x}}$$

$$\Delta y = \sqrt{\frac{1}{g} \frac{L}{N_y}}$$

Pour le pas de temps, comme la grille est carrée, nous pouvons écrire :

$$\Delta t \leq \frac{\Delta x^2}{4}$$

$$\implies \Delta t = \frac{1}{4g} \frac{L^2}{N x^2}$$

3.1.2 Paquet d'onde initial

Pour le paquet d'onde initial nous reprenons la gaussienne de la partie 2.1.2. Nous avons donc :

$$\psi = \psi_0 e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

Où nous considérons $\sigma = \sigma_x = \sigma_y$, l'incertitude sur la position ou l'étalement du condensat.

Cependant, pour faire apparaître un vortex dans ce condensat il faut modifier les conditions initiales en ajoutant une phase au paquet d'onde, nous avons donc :

$$\psi = \psi_0 e^{-\frac{x^2+y^2}{2\sigma^2}} e^{i\theta}$$

Dans notre cas nous pouvons écrire que :

$$\theta = \text{atan2}(y - y_0, x - x_0)$$

Où (x_0, y_0) sont les coordonnées du centre du vortex et où $\text{atan2}(y, x)$ est l'angle entre le plan supérieur et le point (x, y) .

3.2 Traduction en python

Ainsi nous pouvons traduire notre programme en python :
Nous définissons les matrices A et B :

```
1  """Creation des matrice A et B"""
2  Principale_DiagA = (1 + 2*rx + 2*ry + 0.5j*dt*V0) * np.ones(N) #diagonale principale de A
3  Diags_Inf_SuppA = -rx * np.ones(N-1) #Diagonales inferieure et superieure de A
4  DiagDistanteA = -ry * np.ones(N-Ny) #Diagonales distantes de A
5  A = diags([DiagDistanteA, Diags_Inf_SuppA, Principale_DiagA, Diags_Inf_SuppA, DiagDistanteA],
6           [Nx, 1, 0, -1, -Nx], format='csc') #création de la matrice creuse A
7
8  Principale_DiagB = (1 - 2*rx - 2*ry - 0.5j*dt*V0) * np.ones(N) #diagonale principale de B
9  Diags_Inf_SuppB = rx * np.ones(N-1) #Diagonales inferieure et superieure de B
10 DiagDistanteB = ry * np.ones(N-Ny) #Diagonales distantes de B
11 B = diags([DiagDistanteB, Diags_Inf_SuppB, Principale_DiagB, Diags_Inf_SuppB, DiagDistanteB],
12           [Nx, 1, 0, -1, -Nx], format='csc') #création de la matrice creuse B
```

Nous définissons les fonctions de calcul de norme, d'énergies et de la construction de la matrice O :

```
1  """Fonction calculant la norme de la fonction d'onde psi"""
2  def calculate_norm(psi2D):
3      return np.sum(np.abs(psi2D.reshape(Nx, Ny))**2) * dx * dy
4
5  """Fonction calculant l'énergie de la fonction psi"""
6  def calculate_energy(psi2D, psi_prev):
7      p_extrap = (1.5 * psi2D - 0.5 * psi_prev).reshape(Nx, Ny)
8      density = np.abs(p_extrap)**2
9
10     d2x = np.gradient(np.gradient(p_extrap, dx, axis=1), dx, axis=1)
11     d2y = np.gradient(np.gradient(p_extrap, dy, axis=0), dy, axis=0)
12     laplacian = d2x + d2y
13
14     Ec = 0.5 * np.sum(-np.real(np.conj(p_extrap) * laplacian)) * dx * dy #énergie cinétique
15     Ep = np.sum(V * density) * dx * dy #énergie potentielle
16     Ei = 0.5*g * np.sum(density**2) * dx * dy #énergie d'interaction
17
18     return np.real(Ec+Ep+Ei) #énergie totale du condensat
19
20 """Fonction Calculant le moment angulaire de la fonction d'onde psi"""
21 def calculate_angular_momentum(psi2D):
22     p = psi2D.reshape(Nx, Ny) #reshape en 2D pour le calcul des gradients
23
24     gradx = np.gradient(p, dx, axis=1) #gradient de psi selon x
25     grady = np.gradient(p, dy, axis=0) #gradient de psi selon y
```

```

26
27     i = np.conj(p) * (-1j) * (X * grady - Y * gradx) #intégrande
28
29     return np.real(np.sum(i* dx* dy))
30
31 """Fonction qui construit la matrice 0"""
32 def construct_0(psi2D, psi_prev):
33     avg_density = np.abs(1.5 * psi2D - 0.5 * psi_prev)**2
34     #densité extrapolée à mi-pas pour le terme non-linéaire
35     return diags([0.5j * dt * g * avg_density], [0], format='csc')
36     #diagonale du terme d'interaction semi-implicite

```

Maintenant nous pouvons traduire le corps de l'algorithme

```

1 """Boucle d'animation"""
2 def animate(i):
3     global psi2D, phys_time #variables globales modifiées à chaque frame
4
5     for _ in range(step):
6         psi_prev = psi2D.copy() #sauvegarde de l'itération précédente
7
8         0 = construct_0(psi2D, psi_prev)
9         psi_next = spsolve((A + 0), (B - 0) @ psi2D) #résolution du système linéaire creux
10
11         psi2D = psi_next #mise à jour de la fonction d'onde
12         phys_time += dt #avancement du temps physique
13
14         density_2d = np.abs(psi2D.reshape(Nx, Ny))**2 #densité 2D à afficher
15         phase = np.angle(psi2D.reshape(Nx, Ny)) #phase 2D à afficher
16
17         norms.append(calculate_norm(psi2D)) #enregistrement de la norme
18         energies.append(calculate_energy(psi2D, psi_prev)) #enregistrement de l'énergie
19         angular_momentums.append(calculate_angular_momentum(psi2D)) #enregistrement du moment cinétique
20         times.append(phys_time) #temps physique
21
22         im.set_array(density_2d) #mise à jour de l'image de densité
23         im.set_clim(0, np.max(density_2d)) #renormalisation dynamique de la colormap
24         ang.set_array(phase) #mise à jour de l'image de phase
25         ang.set_clim(-np.pi, np.pi) #limites fixes de la colormap de phase
26
27         Norm.set_data(times, norms) #mise à jour de la courbe de norme
28         Energy.set_data(times, energies) #mise à jour de la courbe d'énergie
29         AngMom.set_data(times, angular_momentums) #mise à jour de la courbe de moment cinétique
30
31 """Réglage des axes"""
32 ax3.set_ylim(0, 2 * np.max(norms))
33 ax4.set_ylim(0, 2 * np.max(energies))

```

```

34     ax5.set_ylim(0, 2 * np.max(angular_momentums))
35     ax3.set_xlim(0, 2 * np.max(times))
36     ax4.set_xlim(0, 2 * np.max(times))
37     ax5.set_xlim(0, 2 * np.max(times))
38
39     return [im, ang, Norm, Energy, AngMom] #retour des variables mises à jour

```

3.3 Résultats d'un système Bidimensionnel

3.3.1 Vortex solitaire centré en (0,0)

Nous effectuons la simulation avec les mêmes valeurs de g qu'à la partie 2.3.

Nous lançons le programme pour notre paquet d'onde initial avec une interaction entre les "bosons", g à 1. Pour pouvoir apercevoir le système on réduit la taille physique de la grille ($L = 6$) et l'étalement du condensat ($\sigma = 2$)

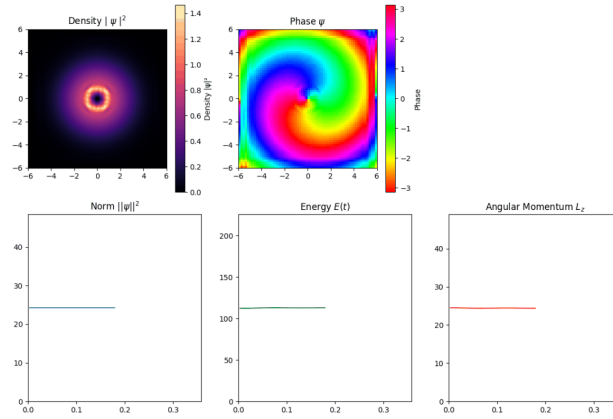


FIGURE 5 – $g = 1$

L'énergie, la norme et le moment angulaire sont bien des quantités conservées pour de faibles interactions entre "bosons". Nous voyons bien le vortex centré en (0,0). Nous pouvons effectuer une nouvelle fois la simulation pour $g = 10$, $L = 10$ et $\sigma = 3$:

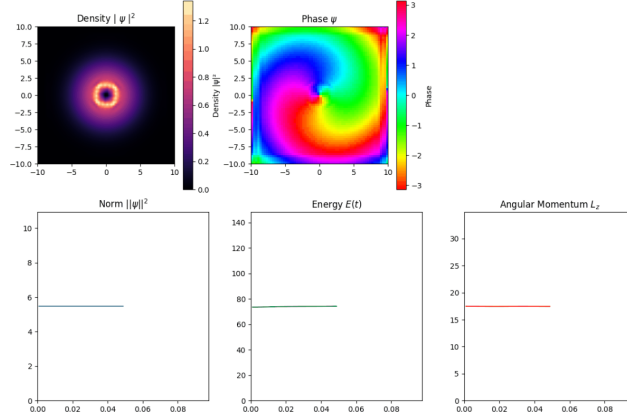


FIGURE 6 – $g = 10$

Moyennant quelques perturbations liées à l'accumulation d'erreurs, l'énergie la norme et le moment angulaire sont bien des quantités conservées pour des interactions un peu plus fortes entre "bosons". Nous voyons toujours bien le vortex centré en $(0,0)$. Nous pouvons effectuer une nouvelle fois la simulation pour $g = 100$, $L = 20$ et $\sigma = 4$:

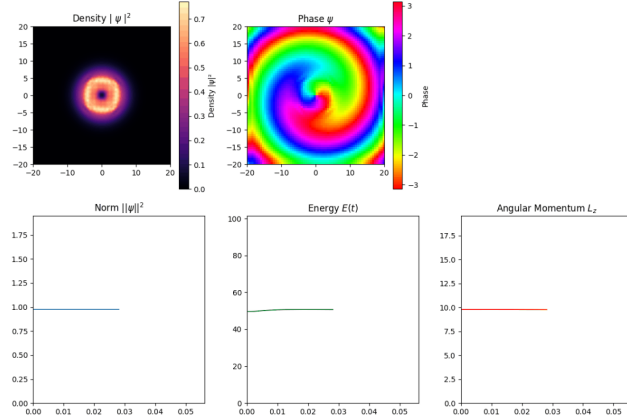


FIGURE 7 – $g = 100$

L'erreur s'accumule de plus en plus, cependant l'énergie la norme et le moment angulaire restent quand même des quantités conservées pour de fortes interactions entre "bosons". Nous voyons une fois de plus le vortex centré en $(0,0)$. Nous pouvons effectuer une nouvelle fois la simulation pour $g = 400$, $L = 30$ et $\sigma = 4$:

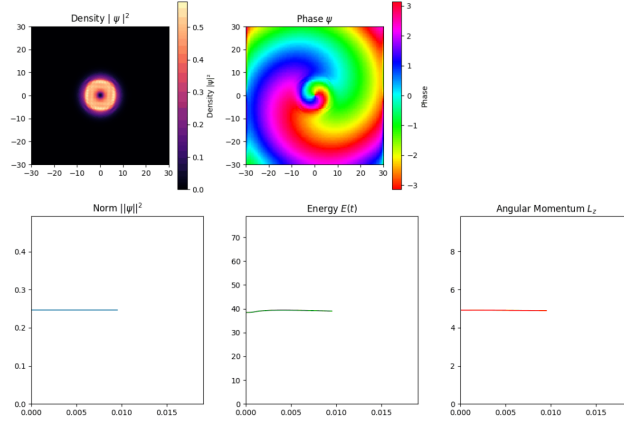


FIGURE 8 – $g = 400$

Pour $g = 400$, l'erreur s'accumule de manière évidente. La norme et le moment angulaire restent quand même des quantités conservées pour de très fortes interactions entre "bosons". La source de l'erreur peut être l'approximation de la densité utilisée. Nous voyons une fois de plus le vortex centré en $(0,0)$. De plus un changement de profil de densité peut être un moyen de réduire l'erreur. (voir annexe 4, partie 5.4, page 30)

3.3.2 Deux vortex centrés en $(-2,2)$ et $(2,-2)$

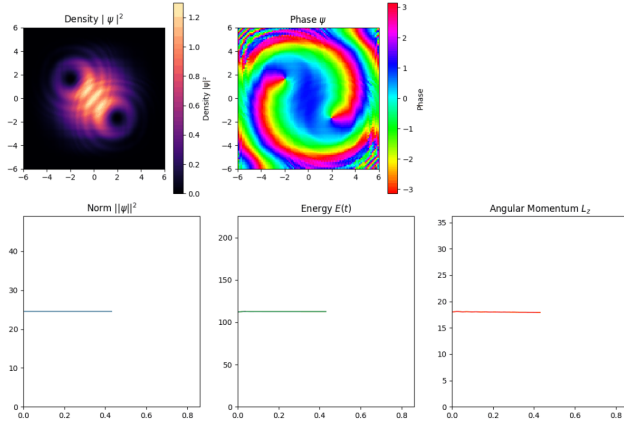


FIGURE 9 – $g = 1$

Pour une faible interaction, nous voyons bien le profil de gaussienne avec les deux trous de densité séparés et une phase tournante autour des singularités.

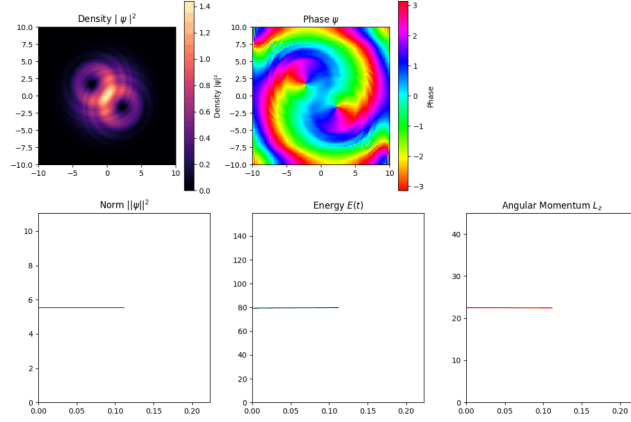


FIGURE 10 – $g = 10$

Pour $g = 10$, les interactions deviennent plus significatives. Nous observons une légère déformation des structures de densité, signe que les deux vortex commencent à interagir de manière plus prononcée. Les quantités restes conservées.

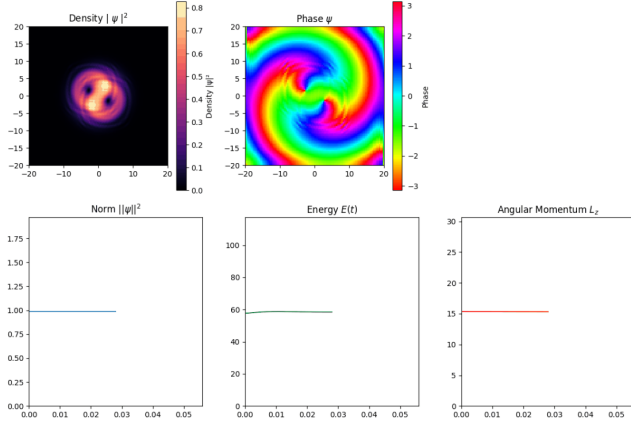


FIGURE 11 – $g = 100$

Pour $g = 100$, les interactions deviennent beaucoup plus significatives. Les quantités testées moyennant quelques erreurs numériques, restent conservées

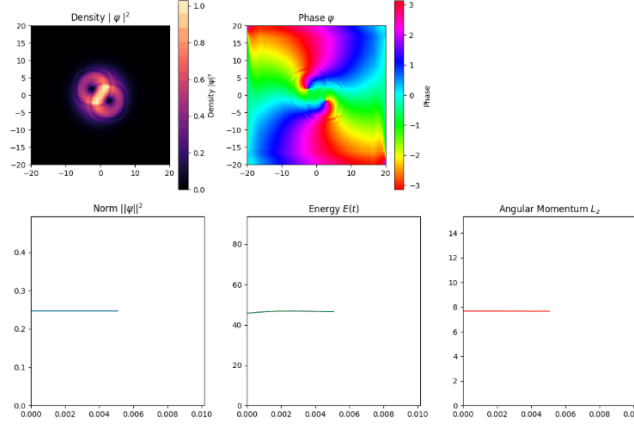


FIGURE 12 – $g = 400$

Enfin pour $g = 400$, les interactions deviennent très fortes. Les quantités testées accumulent de l'erreur numérique, mais restent suffisamment conservées.

4 Conclusion

Nous avons pu, au travers de la résolution numérique de l'équation de Gross-Pitaeski et par l'intermédiaire du langage de programmation Python, effectuer premièrement une étude unidimensionnelle d'un condensat de Bose-Einstein. Cette étude a validée la robustesse de l'utilisation de l'algorithme de Crank-Nicolson dans un cas simple avant de poursuivre vers un système bidimensionnel plus complexe.

D'une part, l'algorithme de Crank-Nicolson, en moyennant les méthodes d'Euler progressive et régressive, garantit une bonne stabilité numérique et une conservation de la norme et de l'énergie. Cette conservation a été vérifiée pour différentes valeurs du paramètre d'interaction : g confirmant la validité physique des simulations sur une large gamme d'interactions.

D'autre part, le passage à un système bidimensionnel a introduit une nouvelle quantité conservée, le moment angulaire. Mais a aussi introduit une complexité supplémentaire sur le plan numérique, avec des matrices creuses de plus grande dimension avec des termes de bord à traiter.

Ce travail pose ainsi les bases techniques nécessaires à des études plus avancées, notamment la simulation complète de la dynamique de vortex, qui constituera le cœur des recherches menées lors de mon stage à l'Université de Purdue dans l'Indiana.

Cependant des améliorations restent possibles : un traitement plus rigoureux des conditions aux bords par l'utilisation de masque ou de potentiels absorbants ou encore l'optimisation de l'algorithme pour plus de rapidité de calcul pour les grandes grilles physiques.

Références

- [1] Wikipédia, *Condensat de Bose-Einstein*,
https://fr.wikipedia.org/wiki/Condensat_de_Bose-Einstein
- [2] La Physique Autrement, *animations quantiques : condensation de Bose-Einstein*,
<https://www.youtube.com/watch?v=6joDoSeq8eg>
- [3] Garaud J., *Time Crystal BEC*, Institut Denis Poisson,
<https://arxiv.org/pdf/2010.04549>
- [4] Den of Learning, *Crank-Nicolson : Second Order 1D Heat Equation Solver*,
<https://www.youtube.com/watch?v=8-LFC3wkphA>
- [5] Wikipédia, *Équation de Gross-Pitaevskii*,
https://fr.wikipedia.org/wiki/%C3%89quation_de_Gross-Pitaevskii
- [6] Hunter J.D., *Matplotlib Documentation*,
<https://matplotlib.org/stable/index.html>
- [7] *SciPy Documentation*,
<https://docs.scipy.org/doc/scipy/>
- [8] Artmenlope, *Solving the 2D Schrödinger equation using the Crank-Nicolson method*,
<https://artmenlope.github.io/solving-the-2d-schrodinger-equation-using-the-crank-nicolson-method/>
- [9] Ajaib M.A., *Numerical Methods and Causality in Physics*, arXiv :1302.5601,
<https://arxiv.org/pdf/1302.5601>

5 Annexes

5.1 Annexe 1 : Etablissement de l'algorithme de Crank-Nicolson 1D

Si le problème est de la forme :

$$\frac{\partial u}{\partial t} = F\left(u, x, t, \frac{\partial u}{\partial x}, \frac{\partial^2 u}{\partial x^2}\right) \quad (8)$$

avec $u = u(x, t)$

En évaluant la fonction u à un instant t et un point x nous pouvons écrire :

$$u_x^t = u(x\Delta x, t\Delta t) \quad (9)$$

- Δx le pas d'espace
- Δt le pas de temps

La méthode de Crank-Nicolson est définie par une moyenne des deux méthodes d'Euler, la méthode progressive et la méthode régressive :

Méthode	Expression
Euler Progressive	$\frac{u_x^{t+\Delta t} - u_x^t}{\Delta t} = F_x^t\left(u, x, t, \frac{\partial u}{\partial x}, \frac{\partial^2 u}{\partial x^2}\right)$
Euler Régressive	$\frac{u_x^{t+\Delta t} - u_x^t}{\Delta t} = F_x^{t+\Delta t}\left(u, x, t, \frac{\partial u}{\partial x}, \frac{\partial^2 u}{\partial x^2}\right)$
Crank-Nicolson	$\frac{u_x^{t+\Delta t} - u_x^t}{\Delta t} = \frac{1}{2} \left(F_x^{t+\Delta t}\left(u, x, t, \frac{\partial u}{\partial x}, \frac{\partial^2 u}{\partial x^2}\right) + F_x^t\left(u, x, t, \frac{\partial u}{\partial x}, \frac{\partial^2 u}{\partial x^2}\right) \right)$

TABLE 1 – Comparaison des méthodes

Dans le cas de notre système nous pouvons réécrire :

$$F(\psi, x, t, \frac{\partial \psi}{\partial x}, \frac{\partial^2 \psi}{\partial x^2}) = -\frac{1}{2} \frac{\partial^2 \psi}{\partial x^2} + \left[V(x) + g|\psi|^2 \right] \psi \quad (10)$$

Nous avons :

$$i \frac{\partial \psi}{\partial t} = F(\psi, x, t, \frac{\partial \psi}{\partial x}, \frac{\partial^2 \psi}{\partial x^2}) \quad (11)$$

$$\frac{\partial \psi}{\partial t} = \frac{1}{i} F(\psi, x, t, \frac{\partial \psi}{\partial x}, \frac{\partial^2 \psi}{\partial x^2}) \quad (12)$$

$$\frac{\partial \psi}{\partial t} = -i F(\psi, x, t, \frac{\partial \psi}{\partial x}, \frac{\partial^2 \psi}{\partial x^2}) \quad (13)$$

Si maintenant nous choisissons d'appliquer à notre système de condensat unidimensionnel l'algorithme de Crank-Nicolson pour en trouver une solution nous pouvons commencer par discrétiser le temps et écrire :

$$\frac{\partial \psi}{\partial t} = \frac{\psi_x^{t+\Delta t} - \psi_x^t}{\Delta t} \quad (14)$$

En reprenant le principe de l'algorithme nous pouvons écrire

$$\frac{\psi_x^{t+\Delta t} - \psi_x^t}{\Delta t} = \frac{-i}{2} \left(F_x^{t+\Delta t} \left(\psi, x, t, \frac{\partial \psi}{\partial x}, \frac{\partial^2 \psi}{\partial x^2} \right) + F_x^t \left(\psi, x, t, \frac{\partial \psi}{\partial x}, \frac{\partial^2 \psi}{\partial x^2} \right) \right) \quad (15)$$

Soit pour notre cas nous pouvons écrire pour la fonction F :

$$F_x^{t+\Delta t} \left(\psi, x, t, \frac{\partial \psi}{\partial x}, \frac{\partial^2 \psi}{\partial x^2} \right) = -\frac{1}{2} \frac{\psi_{x+\Delta x}^{t+\Delta t} - 2\psi_x^{t+\Delta t} + \psi_{x-\Delta x}^{t+\Delta t}}{\Delta x^2} + \left[V(x) + g|\psi|^2 \right] \psi_x^{t+\Delta t}$$

$$F_x^t \left(\psi, x, t, \frac{\partial \psi}{\partial x}, \frac{\partial^2 \psi}{\partial x^2} \right) = -\frac{1}{2} \frac{\psi_{x+\Delta x}^t - 2\psi_x^t + \psi_{x-\Delta x}^t}{\Delta x^2} + \left[V(x) + g|\psi|^2 \right] \psi_x^t$$

En prenant la moyenne et en posant $r = \frac{\Delta t}{2\Delta x^2}$ nous avons :

$$\begin{aligned} \psi_x^{t+\Delta t} - \psi_x^t &= \frac{ir}{2} \left(\psi_{x+\Delta x}^{t+\Delta t} - 2\psi_x^{t+\Delta t} + \psi_{x-\Delta x}^{t+\Delta t} \right) - \frac{i\Delta t}{2} \left(\left[V(x) + g|\psi|^2 \right] \psi_x^{t+\Delta t} \right) \\ &\quad + \frac{ir}{2} \left(\psi_{x+\Delta x}^t - 2\psi_x^t + \psi_{x-\Delta x}^t \right) - \frac{i\Delta t}{2} \left(\left[V(x) + g|\psi|^2 \right] \psi_x^t \right) \end{aligned}$$

En posant $V(x) + g|\psi|^2 = \tilde{V}$ nous pouvons récrire cette équation :

$$\begin{aligned} \psi_x^{t+\Delta t} - \frac{ir}{2} \left(\psi_{x+\Delta x}^{t+\Delta t} - 2\psi_x^{t+\Delta t} + \psi_{x-\Delta x}^{t+\Delta t} \right) &+ \frac{i\Delta t}{2} \tilde{V} \psi_x^{t+\Delta t} \\ &= \frac{ir}{2} \left(\psi_{x+\Delta x}^t - 2\psi_x^t + \psi_{x-\Delta x}^t \right) - \frac{i\Delta t}{2} \tilde{V} \psi_x^t + \psi_x^t \end{aligned}$$

Nous pouvons donc développer et réduire cette expression :

$$\begin{aligned} & -\frac{ir}{2} \psi_{x+\Delta x}^{t+\Delta t} + (1 + ir + \frac{i\Delta t}{2} \tilde{V}) \psi_x^{t+\Delta t} - \frac{ir}{2} \psi_{x-\Delta x}^{t+\Delta t} \\ &= \frac{ir}{2} \psi_{x+\Delta x}^t + (1 - ir - \frac{i\Delta t}{2} \tilde{V}) \psi_x^t + \frac{ir}{2} \psi_{x-\Delta x}^t \end{aligned}$$

Cette expression peut s'écrire sous la forme matricielle telle que :

$$A' \psi_x^{t+\Delta t} = B' \psi_x^t \quad (16)$$

A' et B' ici sont des matrices tridiagonales telles que :

$$A' = \begin{pmatrix} (1 + ir + \frac{i\Delta t}{2}\tilde{V}) & -\frac{ir}{2} & \dots & 0 \\ -\frac{ir}{2} & (1 + ir + \frac{i\Delta t}{2}\tilde{V}) & \ddots & \vdots \\ \vdots & \ddots & \ddots & -\frac{ir}{2} \\ 0 & 0 & -\frac{ir}{2} & (1 + ir + \frac{i\Delta t}{2}\tilde{V}) \end{pmatrix}$$

Et :

$$B' = \begin{pmatrix} (1 - ir - \frac{i\Delta t}{2}\tilde{V}) & \frac{ir}{2} & \dots & 0 \\ \frac{ir}{2} & (1 - ir - \frac{i\Delta t}{2}\tilde{V}) & \ddots & \vdots \\ \vdots & \ddots & \ddots & \frac{ir}{2} \\ 0 & 0 & \frac{ir}{2} & (1 - ir - \frac{i\Delta t}{2}\tilde{V}) \end{pmatrix}$$

Avec :

$$- r = \frac{\Delta t}{2\Delta x^2}$$

$$- \tilde{V} = V(x) + g|\psi|^2$$

Cependant pour le terme non linéaire $|\psi|^2$ nous devons extrapoler grâce à l'itération précédente de la fonction d'onde $\psi_x^{t-\Delta t}$

$$\text{On écrit } |\psi|^2 = \left| \frac{3\psi_x^t - \psi_x^{t-\Delta t}}{2} \right|^2$$

A cette étape nous faisons le choix d'appliquer l'approximation sur le terme non linéaire de la densité uniquement.

Nous pouvons donc scinder les matrices A' et B' tel que :

$$\begin{aligned} A' &= \begin{pmatrix} (1 + ir + \frac{i\Delta t}{2}\tilde{V}) & -\frac{ir}{2} & \dots & 0 \\ -\frac{ir}{2} & (1 + ir + \frac{i\Delta t}{2}\tilde{V}) & \ddots & \vdots \\ \vdots & \ddots & \ddots & -\frac{ir}{2} \\ 0 & 0 & -\frac{ir}{2} & (1 + ir + \frac{i\Delta t}{2}\tilde{V}) \end{pmatrix} = \\ &\begin{pmatrix} (1 + ir + \frac{i\Delta t}{2}(V(x) + g|\psi|^2)) & -\frac{ir}{2} & \dots & 0 \\ -\frac{ir}{2} & (1 + ir + \frac{i\Delta t}{2}(V(x) + g|\psi|^2)) & \ddots & \vdots \\ \vdots & \ddots & \ddots & -\frac{ir}{2} \\ 0 & 0 & -\frac{ir}{2} & (1 + ir + \frac{i\Delta t}{2}(V(x) + g|\psi|^2)) \end{pmatrix} = \\ &\begin{pmatrix} (1 + ir + \frac{i\Delta t}{2}V(x) + \frac{i\Delta t}{2}g|\psi|^2) & -\frac{ir}{2} & \dots & 0 \\ -\frac{ir}{2} & (1 + ir + \frac{i\Delta t}{2}V(x) + \frac{i\Delta t}{2}g|\psi|^2) & \ddots & \vdots \\ \vdots & \ddots & \ddots & -\frac{ir}{2} \\ 0 & 0 & -\frac{ir}{2} & (1 + ir + \frac{i\Delta t}{2}V(x) + \frac{i\Delta t}{2}g|\psi|^2) \end{pmatrix} = \end{aligned}$$

$$\begin{pmatrix} (1+ir+\frac{i\Delta t}{2}V(x)) & -\frac{ir}{2} & \dots & 0 \\ -\frac{ir}{2} & (1+ir+\frac{i\Delta t}{2}V(x)) & \ddots & \vdots \\ \vdots & \ddots & \ddots & -\frac{ir}{2} \\ 0 & 0 & -\frac{ir}{2} & (1+ir+\frac{i\Delta t}{2}V(x)) \end{pmatrix} + \begin{pmatrix} \frac{i\Delta t}{2}g|\psi|^2 & 0 & \dots & 0 \\ 0 & \frac{i\Delta t}{2}g|\psi|^2 & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0 \\ 0 & 0 & 0 & \frac{i\Delta t}{2}g|\psi|^2 \end{pmatrix}$$

Donc nous avons :

$$A' = A + O$$

De la même manière nous avons pour B' :

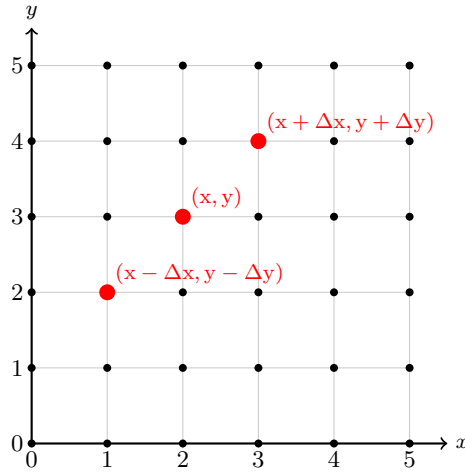
$$B' = B - O$$

Nous pouvons réécrire l'équation matricielle :

$$\begin{aligned} A'\psi_x^{t+\Delta t} &= B'\psi_x^t \\ (A+O)\psi_x^{t+\Delta t} &= (B-O)\psi_x^t \\ \psi_x^{t+\Delta t} &= (A+O)^{-1}(B-O)\psi_x^t \end{aligned}$$

5.2 Annexe 2 : Etablissement de l'algorithme de Crank-Nicolson 2D

Nous effectuons un pavage carré de l'espace. Cette grille sert de base pour discretiser l'espace pour effectuer l'algorithme de Crank-Nicolson.



Ainsi nous pouvons appeler $\psi_{i,j}^t$ évaluée au temps t et la coordonnée (i, j)

Nous pouvons donc écrire pour le temps et les deux coordonnées d'espace :

$$\begin{aligned} i \frac{\partial \psi(x, y, t)}{\partial t} &= i \frac{\psi_{x,y}^{t+\Delta t} - \psi_{x,y}^t}{\Delta t} \\ \frac{\partial^2 \psi(x, y, t)}{\partial x^2} &= \frac{1}{2(\Delta x)^2} \left((\psi_{x+\Delta x, y}^{t+\Delta t} - 2\psi_{x,y}^{t+\Delta t} + \psi_{x-\Delta x, y}^{t+\Delta t}) + (\psi_{x+\Delta x, y}^t - 2\psi_{x,y}^t + \psi_{x-\Delta x, y}^t) \right) \\ \frac{\partial^2 \psi(x, y, t)}{\partial y^2} &= \frac{1}{2(\Delta y)^2} \left((\psi_{x, y+\Delta y}^{t+\Delta t} - 2\psi_{x,y}^{t+\Delta t} + \psi_{x, y-\Delta y}^{t+\Delta t}) + (\psi_{x, y+\Delta y}^t - 2\psi_{x,y}^t + \psi_{x, y-\Delta y}^t) \right) \end{aligned}$$

Ainsi que :

$$\tilde{V} = V(x, y) + g|\psi(x, y, t)|^2$$

Nous pouvons donc maintenant substituer dans l'équation de Gross-Pitaevskii :

$$\begin{aligned} i \frac{\psi_{x,y}^{t+\Delta t} - \psi_{x,y}^t}{\Delta t} = & -\frac{1}{2} \left(\frac{1}{2(\Delta x)^2} \left[(\psi_{x+\Delta x,y}^{t+\Delta t} - 2\psi_{x,y}^{t+\Delta t} + \psi_{x-\Delta x,y}^{t+\Delta t}) + (\psi_{x+\Delta x,y}^t - 2\psi_{x,y}^t + \psi_{x-\Delta x,y}^t) \right] \right. \\ & + \frac{1}{2(\Delta y)^2} \left[(\psi_{x,y+\Delta y}^{t+\Delta t} - 2\psi_{x,y}^{t+\Delta t} + \psi_{x,y-\Delta y}^{t+\Delta t}) + (\psi_{x,y+\Delta y}^t - 2\psi_{x,y}^t + \psi_{x,y-\Delta y}^t) \right] \Big) \\ & + \frac{\tilde{V}}{2} (\psi_{x,y}^{t+\Delta t} + \psi_{x,y}^t) \end{aligned}$$

En multipliant des deux cotés par $\frac{\Delta t}{i}$ Nous pouvons poser :

$$\begin{aligned} r_x &= -\frac{\Delta t}{4i(\Delta x)^2} \\ r_y &= -\frac{\Delta t}{4i(\Delta y)^2} \end{aligned}$$

$$\begin{aligned} \psi_{x,y}^{t+\Delta t} - \psi_{x,y}^t = & \left(r_x \left[(\psi_{x+\Delta x,y}^{t+\Delta t} - 2\psi_{x,y}^{t+\Delta t} + \psi_{x-\Delta x,y}^{t+\Delta t}) + (\psi_{x+\Delta x,y}^t - 2\psi_{x,y}^t + \psi_{x-\Delta x,y}^t) \right] \right. \\ & + r_y \left[(\psi_{x,y+\Delta y}^{t+\Delta t} - 2\psi_{x,y}^{t+\Delta t} + \psi_{x,y-\Delta y}^{t+\Delta t}) + (\psi_{x,y+\Delta y}^t - 2\psi_{x,y}^t + \psi_{x,y-\Delta y}^t) \right] \Big) \\ & + \frac{\tilde{V}\Delta t}{2i} (\psi_{x,y}^{t+\Delta t} + \psi_{x,y}^t) \end{aligned}$$

$$\begin{aligned} \psi_{x,y}^{t+\Delta t} - \psi_{x,y}^t = & \left(r_x \left[(\psi_{x+\Delta x,y}^{t+\Delta t} - 2\psi_{x,y}^{t+\Delta t} + \psi_{x-\Delta x,y}^{t+\Delta t}) + (\psi_{x+\Delta x,y}^t - 2\psi_{x,y}^t + \psi_{x-\Delta x,y}^t) \right] \right. \\ & + r_y \left[(\psi_{x,y+\Delta y}^{t+\Delta t} - 2\psi_{x,y}^{t+\Delta t} + \psi_{x,y-\Delta y}^{t+\Delta t}) + (\psi_{x,y+\Delta y}^t - 2\psi_{x,y}^t + \psi_{x,y-\Delta y}^t) \right] \Big) \\ & - \frac{\tilde{V}i\Delta t}{2} (\psi_{x,y}^{t+\Delta t} + \psi_{x,y}^t) \end{aligned}$$

Soit nous pouvons réécrire :

$$\begin{aligned} & -r_y \psi_{x,y+\Delta y}^{t+\Delta t} - r_y \psi_{x,y-\Delta y}^{t+\Delta t} + (1 + 2r_y + 2r_x + \frac{\tilde{V}i\Delta t}{2}) \psi_{x,y}^{t+\Delta t} - r_x \psi_{x+\Delta x,y}^{t+\Delta t} - r_x \psi_{x-\Delta x,y}^{t+\Delta t} \\ & = r_y \psi_{x,y+\Delta y}^t + r_y \psi_{x,y-\Delta y}^t + (1 - 2r_y - 2r_x - \frac{\tilde{V}i\Delta t}{2}) \psi_{x,y}^t + r_x \psi_{x+\Delta x,y}^t + r_x \psi_{x-\Delta x,y}^t \end{aligned}$$

Nous pouvons donc de la même manière qu'à la section 2.1.1, passer sous la forme matricielle telle que :

$$A' \psi_{x,y}^{t+\Delta t} = B' \psi_{x,y}^t$$

Nous posons $a' = (1 + 2r_y + 2r_x + \frac{\tilde{V}i\Delta t}{2})$ et $b' = (1 - 2r_y - 2r_x - \frac{\tilde{V}i\Delta t}{2})$

$$A' = \begin{pmatrix} a' & -r_x & 0 & \cdots & -r_y & 0 & \cdots & 0 \\ -r_x & a' & -r_x & \cdots & 0 & -r_y & \cdots & 0 \\ 0 & -r_x & a' & & & & & -r_y \\ \vdots & \vdots & -r_x & \ddots & & & \ddots & \vdots \\ -r_y & 0 & & & \ddots & & & \\ 0 & -r_y & & & & \ddots & & \\ \vdots & \vdots & & \ddots & & -r_x & & \vdots \\ 0 & 0 & \cdots & \cdots & 0 & \cdots & -r_x & a' \end{pmatrix}$$

$$B' = \begin{pmatrix} b' & -r_x & 0 & \cdots & -r_y & 0 & \cdots & 0 \\ -r_x & b' & -r_x & \cdots & 0 & -r_y & \cdots & 0 \\ 0 & -r_x & b' & & & & & -r_y \\ \vdots & \vdots & -r_x & \ddots & & & \ddots & \vdots \\ -r_y & 0 & & & \ddots & & & \\ 0 & -r_y & & & & \ddots & & \\ \vdots & \vdots & & \ddots & & -r_x & & \vdots \\ 0 & 0 & \cdots & \cdots & 0 & \cdots & -r_x & b' \end{pmatrix}$$

Nous pouvons maintenant scinder les matrices A' et B' . Nous avons :

$$a' = (1 + 2r_y + 2r_x + \frac{\tilde{V}i\Delta t}{2})$$

Or

$$\tilde{V} = V(x, y) + g|\psi(x, y, t)|^2$$

Donc pour le terme diagonal :

$$\begin{aligned} a' &= (1 + 2r_y + 2r_x + \frac{i\Delta t}{2}V(x, y) + \frac{i\Delta t}{2}g|\psi(x, y, t)|^2) \\ &= (1 + 2r_y + 2r_x + \frac{i\Delta t}{2}V(x, y)) + (\frac{i\Delta t}{2}g|\psi(x, y, t)|^2) \\ &= a + o \end{aligned}$$

Nous pouvons écrire de la même manière pour le terme diagonal de B' :

$$b' = b - o$$

$$A' = \begin{pmatrix} a' & -r_x & 0 & \cdots & -r_y & 0 & \cdots & 0 \\ -r_x & a' & -r_x & \cdots & 0 & -r_y & \cdots & 0 \\ 0 & -r_x & a' & & & & & -r_y \\ \vdots & \vdots & -r_x & \ddots & & & & \vdots \\ -r_y & 0 & & & \ddots & & & \\ 0 & -r_y & & & & \ddots & & \\ \vdots & \vdots & & \ddots & & & -r_x & \vdots \\ 0 & 0 & \cdots & \cdots & 0 & \cdots & -r_x & a' \end{pmatrix}$$

$$= \begin{pmatrix} a+o & -r_x & 0 & \cdots & -r_y & 0 & \cdots & 0 \\ -r_x & a+o & -r_x & \cdots & 0 & -r_y & \cdots & 0 \\ 0 & -r_x & a+o & & & & & -r_y \\ \vdots & \vdots & -r_x & \ddots & & & & \vdots \\ -r_y & 0 & & & \ddots & & & \\ 0 & -r_y & & & & \ddots & & \\ \vdots & \vdots & & \ddots & & & -r_x & \vdots \\ 0 & 0 & \cdots & \cdots & 0 & \cdots & -r_x & a+o \end{pmatrix}$$

$$= \begin{pmatrix} a & -r_x & 0 & \cdots & -r_y & 0 & \cdots & 0 \\ -r_x & a & -r_x & \cdots & 0 & -r_y & \cdots & 0 \\ 0 & -r_x & a & & & & & -r_y \\ \vdots & \vdots & -r_x & \ddots & & & & \vdots \\ -r_y & 0 & & & \ddots & & & \\ 0 & -r_y & & & & \ddots & & \\ \vdots & \vdots & & \ddots & & & -r_x & \vdots \\ 0 & 0 & \cdots & \cdots & 0 & \cdots & -r_x & a \end{pmatrix} + \begin{pmatrix} o & 0 & 0 & \cdots & 0 & \cdots & 0 \\ 0 & o & 0 & \cdots & 0 & \cdots & 0 \\ 0 & 0 & o & & & & 0 \\ \vdots & \vdots & 0 & \ddots & & & \vdots \\ 0 & 0 & & & \ddots & & \\ 0 & 0 & & & & \ddots & \\ \vdots & \vdots & & \ddots & & & 0 & \vdots \\ 0 & 0 & \cdots & \cdots & 0 & \cdots & 0 & o \end{pmatrix}$$

Ici aussi nous pouvons écrire une extrapolation de la densité grâce à l'itération précédente :

$$|\psi(x, y, t)|^2 = \left| \frac{3\psi_{x,y}^{t+\Delta t} - \psi_{x,y}^t}{2} \right|^2$$

Ainsi le terme diagonal de O : o s'écrit :

$$o = \frac{i\Delta t}{2} g |\psi(x, y, t)|^2 = \frac{gi\Delta t}{2} \left| \frac{3\psi_{x,y}^{t+\Delta t} - \psi_{x,y}^t}{2} \right|^2 = \frac{gi\Delta t (3\psi_{x,y}^{t+\Delta t} - \psi_{x,y}^t)^2}{8}$$

Donc :

$$A' = A + O$$

Et ainsi par analogie :

$$B' = B - O$$

Avec O la matrice diagonale de terme diagonal o .

Ainsi nous retrouvons l'équation matricielle :

$$\begin{aligned} A'\psi_x^{t+\Delta t} &= B'\psi_x^t \\ (A+O)\psi_x^{t+\Delta t} &= (B-O)\psi_x^t \\ \psi_x^{t+\Delta t} &= (A+O)^{-1}(B-O)\psi_x^t \end{aligned}$$

Il s'agit de la même équation matricielle que dans la partie 2.1.1 page 4 cependant il y a la présence de termes de bords r_y en plus dans les matrices A et B .

5.3 Annexe 3 : Simulation de l'équation de Gross-Pitaevskii 1D avec un soliton lumineux

L'algorithme utilisé sera toujours celui de Crank-Nicolson mais la condition initiale sera différente. Nous prenons la solution sous forme de soliton lumineux soit :

$$\psi(x, t) = \psi(0)e^{-i\mu t/\hbar} \frac{1}{\cosh\left(\sqrt{2m|\mu|/\hbar^2}x\right)}$$

On traduit donc en python dans les conditions initiales :

```

1  """Conditions initiales"""
2  g = -1 #Interaction entre particules repulsives
3  x = np.linspace(-L, L, Nx) #On initialise un vecteur avec des valeurs de x entre -L et L
4  V = np.zeros(Nx) #On choisit un potentiel de piègeage nul
5  v = 1 #Vitesse du soliton
6  psi = (1 / np.cosh(1 * (x + 5))) * np.exp(1j * v * x).astype(complex) #Profil initial
7  psi_prev = psi.copy() #On copie pour traiter la non linéarité

```

Le soliton a une vitesse initiale et nous avons donc :

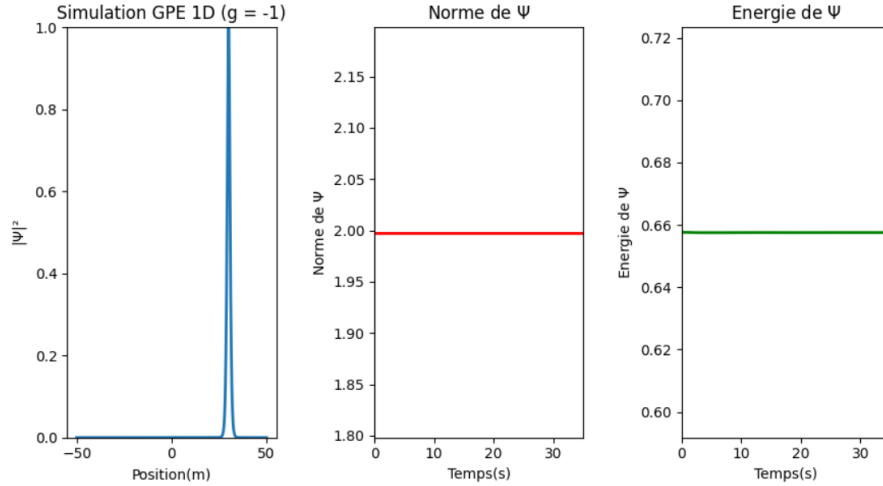


FIGURE 13 – $g = -1$

Le soliton évolue bien dans l'espace sans déformation.

5.4 Annexe 4 : Simulation de la l'équation de Gross-Pitaevskii 2D avec le profil de densité de Thomas-Fermi

Nous avons le profil de densité de Thomas-Fermi :

$$\psi(x, y) = \sqrt{\frac{\mu - V(x, y)}{g}}$$

Avec μ le potentiel chimique, $V(x, y)$ le potentiel harmonique et g l'interaction entre les "bosons". Nous choisissons de fixer le potentiel chimique comme le potentiel harmonique évalué au rayon de Thomas-Fermi $R_{TF} = L/2$.

Nous avons donc :

$$\psi(r) = \max\left(0, \sqrt{\frac{V(R_{TF}) - V(r)}{g}}\right) = \max\left(0, \sqrt{\frac{\frac{1}{2}R_{TF}^2 - \frac{1}{2}r^2}{g}}\right)$$

Nous définissons les conditions initiales :

```

1  """Condition Initiale"""
2  sigma = 1 #largeur du paquet d'onde gaussien initial
3  density_profile = np.maximum(0, ((0.5 * (L/2)**2) - V)/g)
4  theta = np.arctan2(Y, X) #angle centré à l'origine (charge +1)
5  theta2 = np.arctan2(Y+2, X-2) #angle centré en un point (2,-2) (charge +1)
6  theta3 = np.arctan2(Y-2, X+2) #angle centré en un point (-2,2) (charge +1)
7  psi = np.sqrt(density_profile)*np.exp(1j*theta2)*np.exp(1j*theta3).astype(complex)
8  #Profile de Thomas-Fermi
9  psi2D = psi.flatten() #mise à plat en vecteur 1D pour la résolution matricielle

```

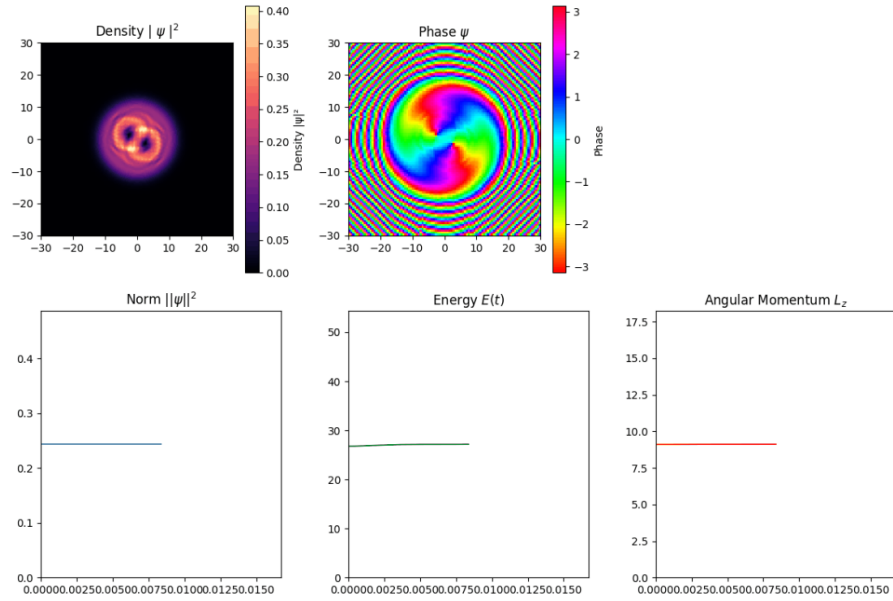


FIGURE 14 – profil de Thomas-Fermi

Le profil de Thomas-Fermi confine le condensat dans un puits de potentiel solide.